CODEALPHA INTERNSHIP

Data Science Design Report

Four reproducible studies

Prepared for **nisa3131**
19 October 2026 · Asia/Dhaka

Standard design report layout approved by the user. Findings use the uploaded datasets and measured outputs from the accompanying projects.

Contents

01 Iris Flower Classification

02 Unemployment Analysis with Python

03 Car Price Prediction with Machine Learning

04 Sales Prediction using Python

The task document requests two or three completed tasks. This portfolio covers all four at the user’s request. GitHub upload and external submission status are tracked separately from the analytical findings.

01 Iris Flower Classification

REPOSITORY CodeAlpha_IrisFlowerClassification

1 Objective

Classify setosa, versicolor, and virginica flowers from sepal length, sepal width, petal length, and petal width. The task covers supervised classification, preprocessing, model comparison, and evaluation on held-out examples.

2 Data and provenance

Input: the user-supplied archive.zip, containing Iris.csv. The task document links to https://www.kaggle.com/datasets/saurabh00007/iriscsv. That link is cited as the assignment's dataset reference; these analyses use the uploaded files rather than a new download. Original collection details and redistribution licensing were not independently verified. No dataset license is asserted.

Attribute	Observed data / treatment
Observations	150; 50 per species
Predictors	Four flower measurements in centimetres; Id excluded
Data quality	No missing values or duplicate complete rows
Repeated measurements	3 repeated measurement rows; 147 unique measurement groups
Duplicate treatment	Retain observations; keep identical measurement groups together in every split

Identical measurements can represent separate flowers, so they are retained. Their group identity is used for both the train/test partition and cross-validation to prevent the same measurement vector appearing on both sides. No identical measurement group has conflicting species labels.

3 Method and evaluation

With random seed 42, the first fold from a shuffled five-fold StratifiedGroupKFold creates 120 training and 30 held-out observations. Each held-out species has 10 examples, and no measurement group crosses the partition. Group constraints make stratification approximate in general.

Four candidates are compared: a most-frequent-class baseline, logistic regression, k-nearest neighbours, and a radial-basis support vector classifier. Numeric scaling is inside each estimator's pipeline and therefore fit only on the training portion of each fold. Predefined grids are C = 0.1, 1, 10 for logistic regression and SVM, and k = 3, 5, 7 for neighbours.

Training-only, five-fold group-aware cross-validation selects hyperparameters using mean macro F1. Model-level ties use mean CV accuracy and then model name. The winner is fit on the full training partition and evaluated once on the held-out partition. The test score does not choose the model.

4 Results and interpretation

Candidate	CV macro F1 mean ± SD	CV accuracy
logistic regression	0.9666 ± 0.0312	0.9667
knn	0.9582 ± 0.0264	0.9583
rbf svm	0.9582 ± 0.0264	0.9583
dummy most frequent	0.1667 ± 0.0000	0.3333

Training cross-validation scores. The standard deviation describes variation across five folds, not a confidence interval.

Selected model: logistic regression (C = 10). Its mean CV macro F1 is 0.9666. Held-out accuracy and macro F1 are 1.000 and 1.000, respectively. The most-frequent baseline reaches 0.333 accuracy and 0.167 macro F1.

Species	Test n	Precision	Recall	F1
setosa	10	1.000	1.000	1.000
versicolor	10	1.000	1.000	1.000
virginica	10	1.000	1.000	1.000

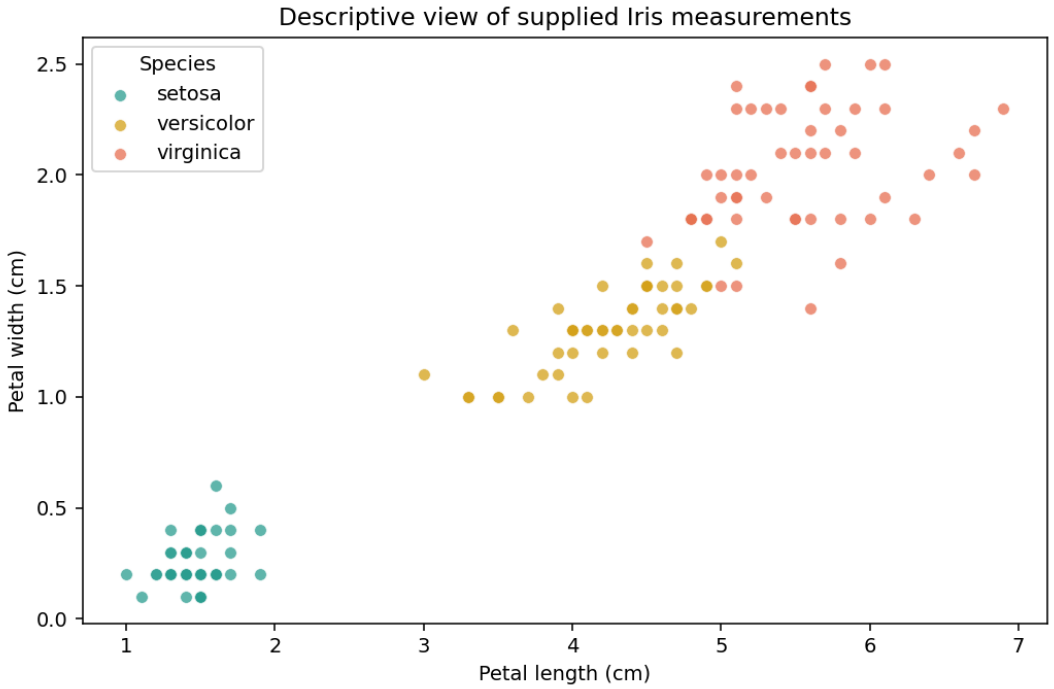


Figure 1. Descriptive petal measurements in the supplied dataset. Setosa is visibly separated; versicolor and virginica overlap more.

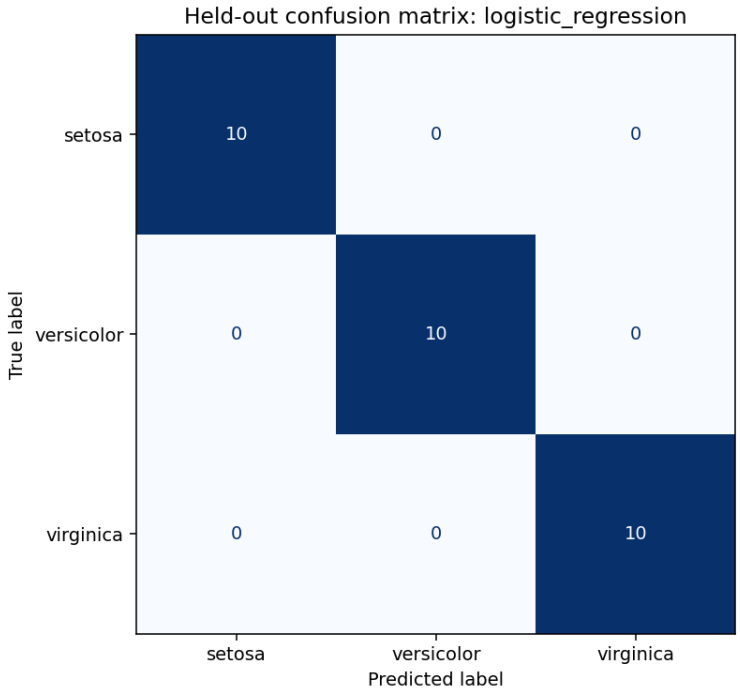


Figure 2. Held-out confusion matrix for the CV-selected model. All 30 test observations are correctly classified in this split.

5 Limitations and next steps

- A perfect score on 30 held-out examples has substantial sampling uncertainty. It does not establish performance on flowers from new locations, instruments, or populations.
- Grouping identical vectors reduces direct duplicate leakage; unrecorded biological or collection relationships may still remain.
- The labels and measurements are not independently verified. This is an educational model, with no external validation or deployment assessment.
- The next evaluation should use an independently collected, well-documented flower dataset and report uncertainty before relying on predictions operationally.

6 Reproducibility

Validation recorded on 19 October 2026: the delivered notebook executed 4 code cells with 0 recorded errors. The analysis pipeline and its built-in data/evaluation checks completed successfully. The project’s 2 unit tests also passed.

Install the pinned requirements and rerun from the repository root. The notebook calls the same analysis pipeline.

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python analysis.py
python -m unittest discover -s tests -v
```

outputs/metrics.json records versions, split sizes, selected parameters, and per-class evaluation. outputs/data_audit.json records the source SHA-256 and quality checks. split_assignments.csv and cv_results.csv expose group membership and every tried configuration; test_predictions.csv records held-out outcomes. The final estimator is fit on training data only.

02 Unemployment Analysis with Python

REPOSITORY CodeAlpha_UnemploymentAnalysis

1 Objective

Clean and explore the supplied unemployment data, describe time and regional patterns around the Covid-19 disruption, assess what the short series can say about seasonality, and identify policy questions supported by the evidence. This is a descriptive analysis.

2 Data and provenance

Input: the user-supplied archive (1).zip, containing Unemployment in India.csv and Unemployment_Rate_upto_11_2020.csv. The task document links to https://www.kaggle.com/datasets/gokulrajkmv/unemployment-in-india. That link is cited as the assignment's dataset reference; these analyses use the uploaded files rather than a new download. Original collection details and redistribution licensing were not independently verified. No dataset license is asserted.

Dataset	Raw / clean rows	Observed coverage
Historical state–area	768 / 740	May 2019–June 2020; 28 states/territories; Rural and Urban observations
2020 state	267 / 267	January–October 2020; 27 states/territories; macro-region labels

The filename ending in 11_2020 does not establish November coverage: the last observed month is October. The two files remain separate because their overlapping dates and different granularities would otherwise create double counting. No rural counterpart is invented for historical Chandigarh, which has only Urban records.

3 Cleaning and comparison design

Strip headers and text, parse monthly dates and numeric values strictly, standardize frequency labels, and rename the repeated Region field to macro_region in the 2020 file. Remove only 28 fully blank historical rows. Do not impute rates. Required fields, finite values, percentage bounds, integral employment counts, unique observation keys, and complete comparison panels are checked.

The historical entity-month grid has 30 missing cells and the 2020 state grid has 3. These are documented rather than filled. Source coordinate labels appear reversed because the latitude-labeled values exceed 90 degrees; they are preserved with explicit source labels and are not used for geographic maps.

The main comparison uses the same 25 complete states/territories across January–March (reference), April–May (initial disruption), and August–October (later period). Jammu & Kashmir and Sikkim are excluded for incomplete coverage. First average months within each state, then average states with equal weight. The balanced monthly trend uses the same states. Historical rural–urban comparisons require 20 common states with both areas observed in every relevant month; the full 14-month trend and January–June period panel are explicitly checked.

All aggregate rates in this report are unweighted state or matched state–area means. They are not official population-weighted national rates. Rate differences are percentage points (pp). The reference period is descriptive; March may already contain pandemic disruption.

4 Results and policy interpretation

Balanced 25-state measure	Jan–Mar	Apr–May	Aug–Oct
Unemployment rate (%)	9.20	23.20	8.90
Labour participation (%)	44.13	37.33	42.07

Equal state weights and the same states in each period.

Mean unemployment rises by **14.00 pp** in April–May and is 0.30 pp below the reference in August–October. The balanced monthly peak is May 2020 at 23.37%. 9 of 25 states remain above their own reference rate in the later period. Participation also falls and does not return to its reference mean, so a lower unemployment percentage alone does not establish full labour-market recovery.

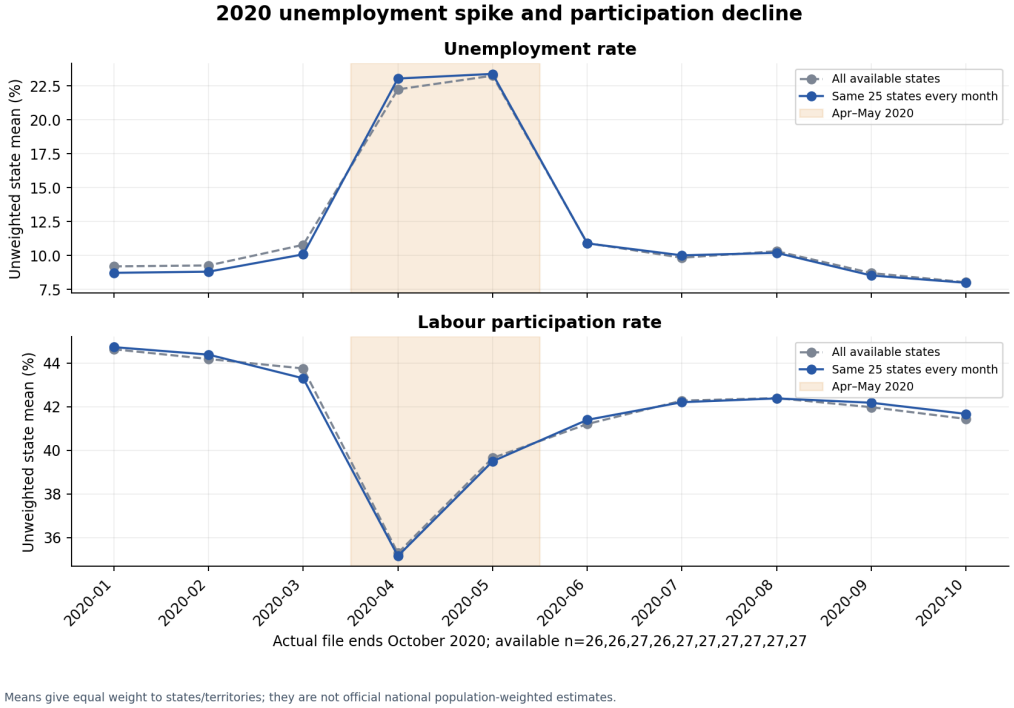


Figure 1. Balanced 2020 state means for unemployment and labour participation. These show timing and association, not a causal pandemic effect.

Largest observed increases	Apr–May minus Jan–Mar (pp)
Puducherry	65.84
Jharkhand	42.93
Tamil Nadu	38.14
Bihar	34.19
Karnataka	21.60

Descriptive equal-period comparisons within each complete state; no survey uncertainty estimates are supplied.

Matched historical area	Jan–Mar %	Apr–May %	June %
Rural	9.48	23.31	12.94
Urban	12.15	27.92	13.83

Separate historical dataset; 20 same states per area in all comparison months.

The matched Urban-minus-Rural gap is 2.67 pp before the disruption, 4.61 pp in April–May, and 0.89 pp in June. These comparisons describe the observed matched states rather than every rural or urban resident.

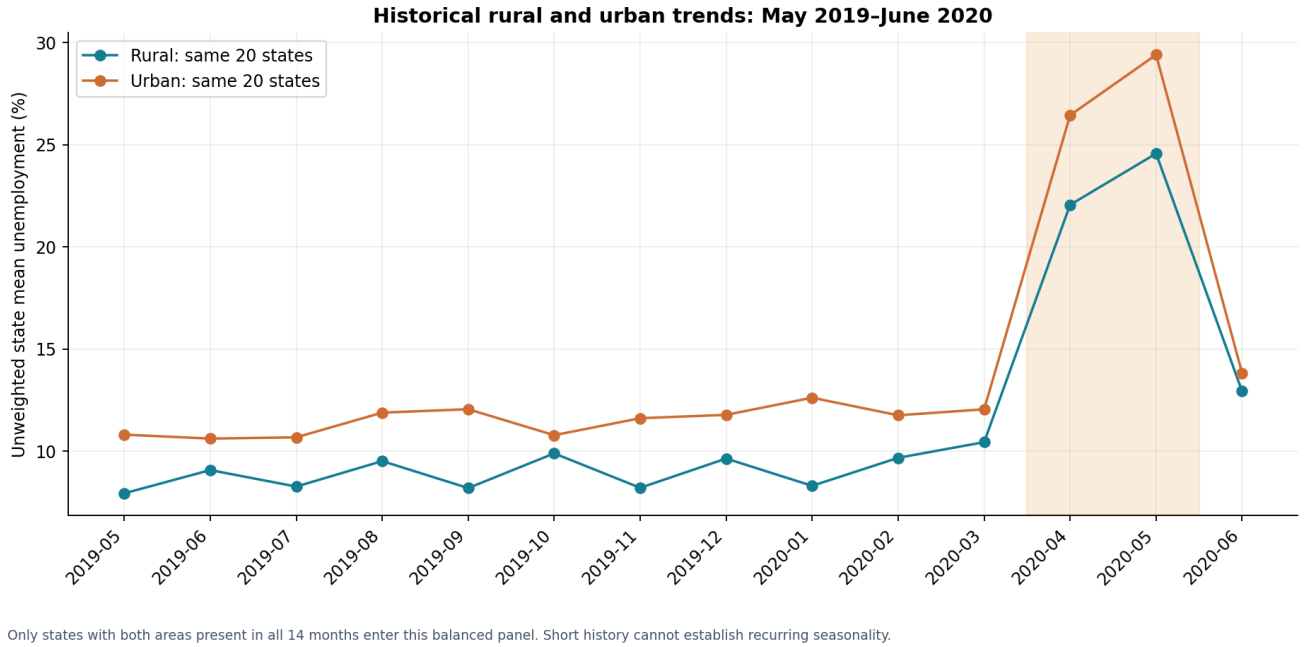


Figure 2. Historical rural and urban means over a common, complete state panel. The short series supports observed fluctuations, not a recurring seasonal cycle.

Within May–December 2019, the balanced rural series is highest in October (9.90%) and lowest in May (7.95%); the urban series is highest in September (12.06%) and lowest in June (10.63%). With only 14 historical and 10 state-series months, these are within-year patterns. Several comparable years are needed to distinguish seasonality from trend, sampling variation, and exceptional disruption.

- Policy direction: monitor unemployment, participation, and employment together. The simultaneous rate rise and participation fall motivate rapid access to income support and employment services during sudden disruptions.
- Use local monitoring and verified affected-labour-force sizes to plan support. Differences across states and rural–urban panels motivate further investigation; an equal-state mean alone is insufficient for allocating national funds.
- Check access to work and income protection in both rural and urban areas. Occupation and migration explanations require additional data because those fields are absent.
- Obtain longer series, survey weights, sample sizes, and policy-exposure information before evaluating a particular program’s effectiveness.

5 Limitations and next steps

There is no control group, identified counterfactual, policy-exposure measure, survey weight, sample size, or standard error. Causality and uncertainty intervals cannot be inferred from these files. Complete-panel

exclusions limit coverage; survey methodology may differ across the two files. No predictive model, seasonal decomposition, official national estimate, or causal pandemic effect is claimed.

6 Reproducibility

Validation recorded on 19 October 2026: the delivered notebook executed 4 code cells with 0 recorded errors. The analysis pipeline and its built-in data/evaluation checks completed successfully.

Install the pinned requirements and rerun from the repository root. The notebook calls the same analysis pipeline.

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python analysis.py
```

outputs/metrics.json records both source SHA-256 hashes, cleaning counts, complete panels, period means, and validation outcomes. Cleaned CSVs, missing-observation tables, entity/month coverage, matched comparisons, and six figures preserve the audit trail. No network is needed for analysis. --data-dir and --output-dir support alternate local paths.

03 Car Price Prediction with Machine Learning

REPOSITORY CodeAlpha_CarPricePrediction

1 Objective

Predict Selling_Price from the supplied historical vehicle features using a regression workflow with cleaning, feature engineering, preprocessing, model selection, and held-out evaluation. Although the assignment calls this car price prediction, the supplied file also includes motorcycles and scooters, so interpretation covers used vehicles.

2 Data and provenance

Input: the user-supplied archive (2).zip, containing car data.csv. The task document links to https://www.kaggle.com/datasets/vijayaadithyanvg/car-price-predictionused-cars. That link is cited as the assignment's dataset reference; these analyses use the uploaded files rather than a new download. Original collection details and redistribution licensing were not independently verified. No dataset license is asserted.

Attribute	Observed data / treatment
Records	301 raw; 2 exact duplicates removed; 299 retained
Vehicle groups	97 normalized vehicle names
Year range	2003–2018
Missing values	None in the supplied file
Inputs	Name, year, present price, distance driven, fuel, selling type, transmission, and owners
Price units	Original dataset units; currency and scale not independently verified

Normalize categorical case and whitespace. Vehicle_Age = 2018 − Year uses the maximum year in this historical file as a fixed reference; it is neither the verified collection date nor the vehicle's age today. Keep the 500,000 km outlier and rare fuel/ownership values rather than removing them silently. Horsepower, brand goodwill, accident history, and condition are absent, so no such fields are invented.

3 Method and evaluation

After deduplication, seeded GroupShuffleSplit holds out 20% of normalized name groups: 229 training rows across 77 names and 70 test rows across 20 names. No name overlaps. The group fraction does not force a 20% row fraction. This evaluates generalization to unseen named vehicle models, which is more demanding than random listings of familiar models.

A ColumnTransformer pipeline applies median imputation and scaling to numeric features, and most-frequent imputation plus one-hot encoding to categorical features. All learned preprocessing is fit within training folds; unknown categories are ignored safely. The numeric features are historical age, present price, distance driven, and owners.

A median baseline, Ridge, random forest, and gradient boosting use the same five training-only GroupKFold splits. Predeclared grids select the lowest mean grouped CV MAE. The winning model and

baseline are then evaluated once on the untouched held-out names. Post-selection test permutation importance is descriptive and does not select or tune the model.

4 Results and interpretation

Candidate	CV MAE	Fold SD
random forest	0.9258	0.4306
gradient boosting	0.9504	0.5795
ridge	1.0617	0.2976
dummy median	3.2631	0.9471

Training-only group-aware comparison. MAE and SD use dataset price units; fold SD is not a confidence interval.

Selected model: random forest, with 300 trees, unrestricted depth, and minimum leaf size 3. Mean grouped CV MAE is 0.9258.

Held-out model	MAE	RMSE	R^2
Selected random forest	2.0562	3.3528	0.5839
Median baseline	3.4954	5.4453	-0.0975

Evaluation on 70 rows from 20 unseen named models. Lower error and higher R^2 are better.

Test MAE is 41.2% lower than the median baseline. The gap between training CV and test errors is material and should temper expectations about transfer to new vehicle groups. MAE is average absolute error; RMSE penalizes larger errors more heavily; R^2 is a split-specific squared-error comparison with the test mean.

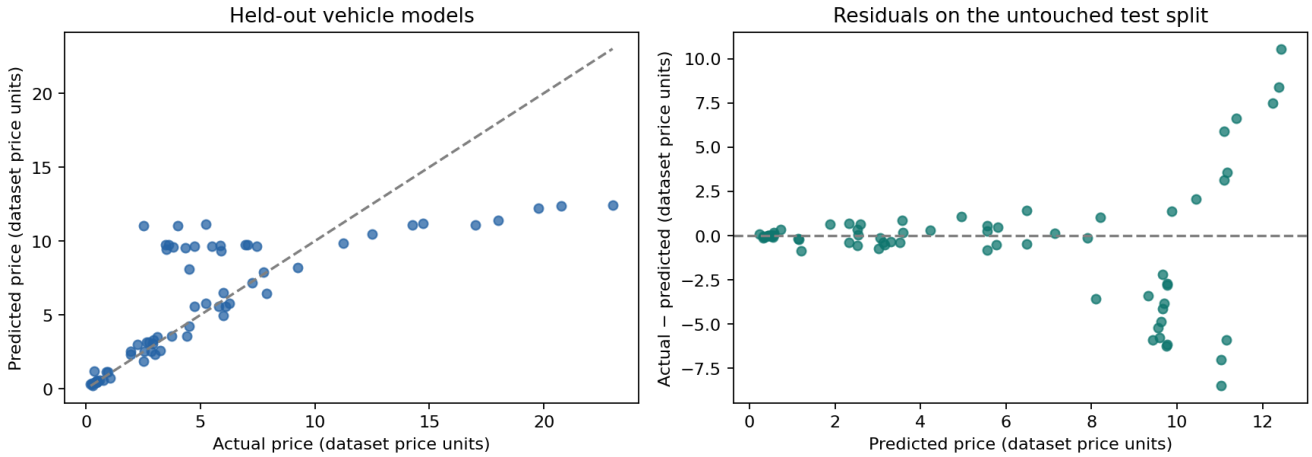


Figure 1. Predictions and residuals for held-out vehicle names. Individual errors remain substantial despite improvement over the baseline.

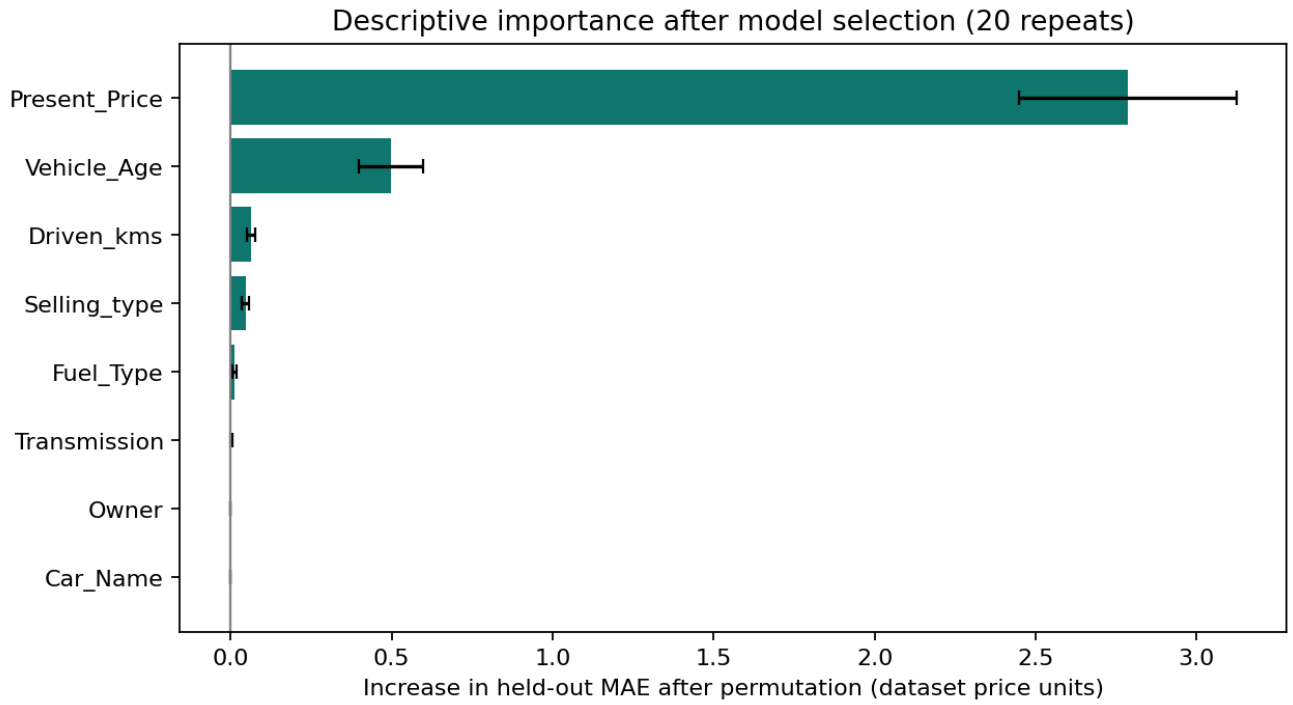


Figure 2. Post-selection importance: increase in held-out MAE after input shuffling, with 20-repeat variation. Correlated inputs may share or mask importance.

Present_Price has the largest measured permutation effect: 2.787 dataset price units of added MAE on average. This is predictive association, not a causal effect. Car_Name importance is zero by construction because every held-out name is unseen and its one-hot categories are unknown. Present_Price would be useful operationally only if its definition and timestamp were verified and the value were available when a prediction is requested.

5 Limitations and next steps

- The historical convenience sample is small and mixes vehicle types. Location, collection date, currency, condition, service history, and accident history are unverified or missing.
- The name-based holdout estimates transfer to unseen names within this file. It does not establish current-market accuracy, transport to a new country, or a production valuation service.
- Grouping and exact deduplication reduce direct leakage; hidden repeated listings under different names cannot be ruled out. One seeded holdout is not a population uncertainty interval.
- Collect representative, dated and permissioned listings, verify units and predictor availability, and define a deployment-specific validation protocol before using predictions for valuation.

6 Reproducibility

Validation recorded on 19 October 2026: the delivered notebook executed 4 code cells with 0 recorded errors. The analysis pipeline and its built-in data/evaluation checks completed successfully. The project’s 4 unit tests also passed.

Install the pinned requirements and rerun from the repository root. The notebook calls the same analysis pipeline.

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python analysis.py
python -m unittest discover -s tests -v
```

outputs/metrics.json and data_audit.json record the source SHA-256, duplicate policy, split counts, versions, parameters, and errors. split_assignments.csv, cv_results.csv, heldout_predictions.csv, and permutation_importance.csv expose the evaluation trail. Four chart PNGs are generated. The saved model.joblib is fit on training data only; only load serialized models from a trusted source.

04 Sales Prediction using Python

REPOSITORY CodeAlpha_SalesPrediction

1 Objective

Predict Sales from TV, Radio, and Newspaper advertising measurements, comparing regression models and exploring how their predictions vary with advertising inputs. The available cross-sectional data support conditional prediction for new observations. Dated future forecasting, target segments, and online platforms cannot be evaluated because those fields are absent.

2 Data and provenance

Input: the user-supplied archive (3).zip, containing Advertising.csv. The task document links to https://www.kaggle.com/datasets/bumba5341/advertisingcsv. That link is cited as the assignment's dataset reference; these analyses use the uploaded files rather than a new download. Original collection details and redistribution licensing were not independently verified. No dataset license is asserted.

Attribute	Observed data / treatment
Observations	200
Predictors / target	TV, Radio, Newspaper / Sales
Data quality	No missing model measurements; no duplicate observations after excluding row index
Identifier	Verified sequential Unnamed: 0 excluded from predictors; retained for row traceability
Units	Observed dataset units; no verified currency or sales-volume scale

No dated campaign, geography, customer segment, platform-detail, price, or macroeconomic controls are supplied. The analysis preserves observed values and does not assign currency, create missing variables, or interpret the index as time.

3 Method and evaluation

With seed 42, a fixed random 80/20 split creates 160 training and 40 untouched test rows. A chronological split is unavailable because there are no dates. All six predeclared candidates use the same five shuffled training-only cross-validation folds. Median imputation, scaling, and interaction/polynomial transformations are inside each fold's pipeline.

Compare a mean baseline, linear regression, Ridge, a TV × Radio interaction model, quadratic Ridge, and random forest using fixed hyperparameters documented in make_models(). Lowest mean training CV RMSE selects the model before the selected estimator and baseline are evaluated once on the held-out rows. The fitted model remains trained on the 160-row training partition; it is not refit on the held-out data.

4 Results and marketing interpretation

Candidate	CV RMSE ± SD	CV MAE	CV R²
Quadratic ridge (alpha=1)	0.671 ± 0.375	0.441	0.977
TV x Radio interaction	0.968 ± 0.370	0.701	0.957
Random forest	1.021 ± 0.247	0.738	0.956
Linear regression	1.676 ± 0.297	1.237	0.870
Ridge (alpha=1)	1.677 ± 0.289	1.237	0.871
Dummy mean	5.150 ± 0.445	4.191	-0.099

Training-only model comparison. Fold SD is variation across five folds, not a confidence interval.

Selected model: Quadratic ridge (alpha=1). The polynomial feature representation allows curvature and interactions, with Ridge regularization applied after scaling.

Held-out model	MAE	RMSE	R²
Selected quadratic Ridge	0.4848	0.6210	0.9878
Mean baseline	4.9625	5.6315	-0.0048

Untouched 40-row test set. MAE/RMSE use observed Sales units; lower is better. Higher R² is better.

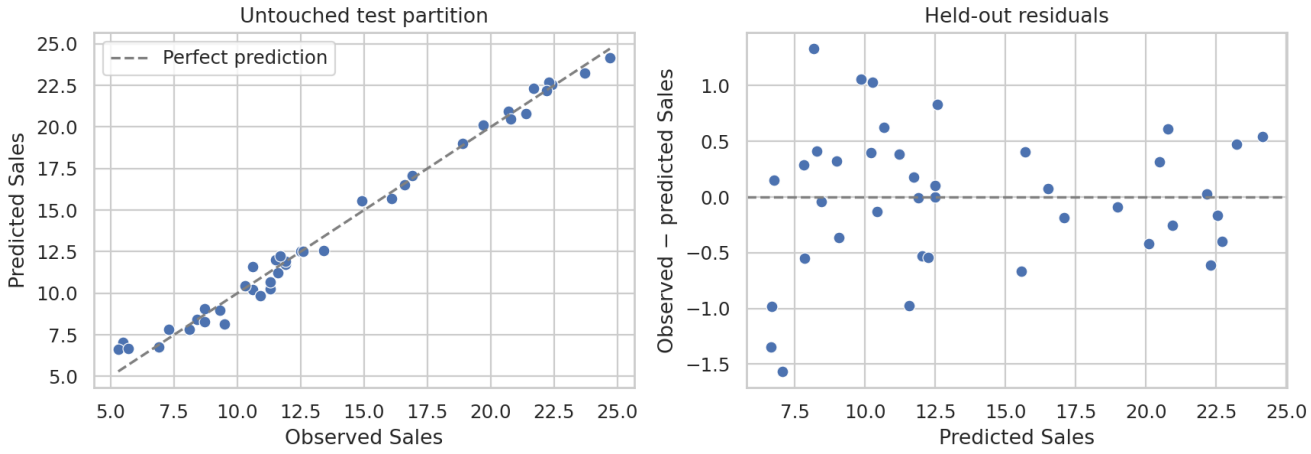


Figure 1. Selected-model held-out predictions and residuals. Strong performance on this split does not establish future-period validity.

Training linear reference	Sales association per 1 SD input increase
TV	3.764
Radio	2.792
Newspaper	0.056

Standardized coefficients from the training-only linear reference model, holding the other observed inputs fixed.

TV and Radio show stronger conditional linear associations than Newspaper in this training sample. The linear reference differs from the selected nonlinear predictor. These coefficients do not establish an intervention effect, and a small Newspaper coefficient does not establish a zero causal contribution.

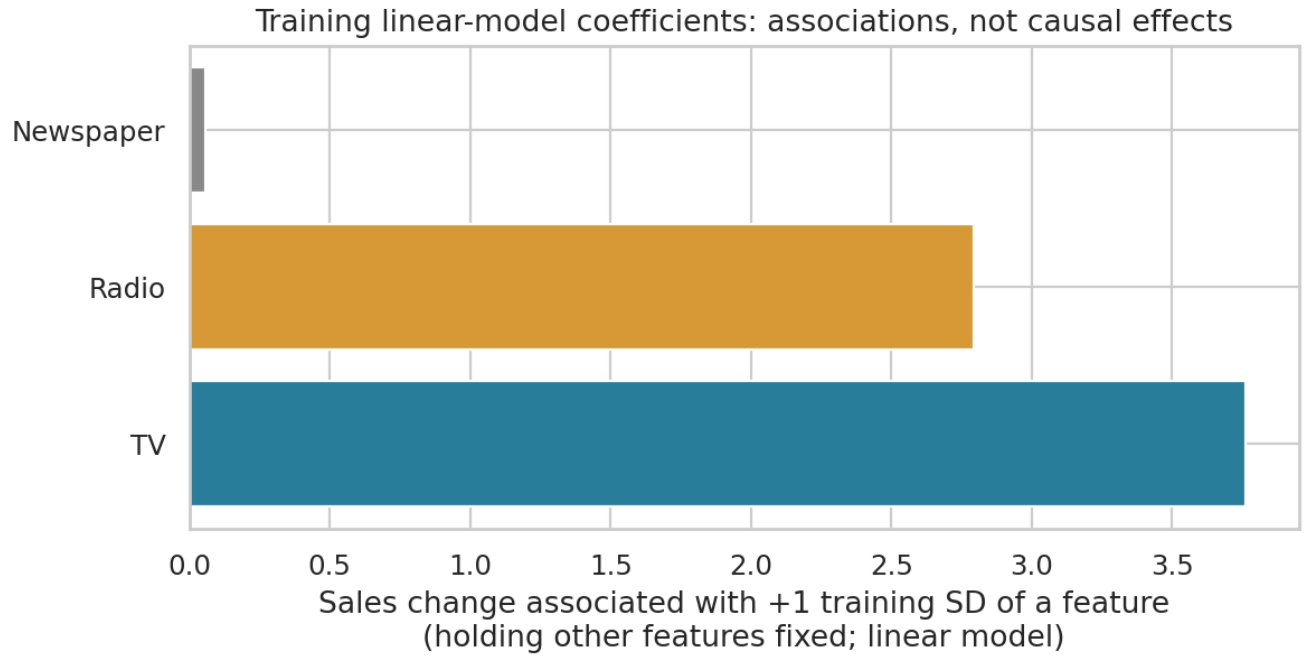


Figure 2. Training-only standardized coefficients. Coefficients summarize fitted associations and must not be read as causal return on advertising.

Hypothetical input scenario	Predicted Sales	Δ from reference
Observed training reference	14.662	+0.000
TV −20% (range clipped)	13.268	-1.394
TV +20% (range clipped)	15.903	+1.241
Radio −20% (range clipped)	13.759	-0.903
Radio +20% (range clipped)	15.570	+0.908
Newspaper −20% (range clipped)	14.643	-0.019
Newspaper +20% (range clipped)	14.681	+0.018

Starting point: actual training source row 20. All values use observed dataset units.

Each sensitivity scenario varies one channel by ±20%, clipped to its observed training range, while holding the others fixed. Marginal range checks do not guarantee joint observational support. There is no observed scenario outcome. These are model-based illustrations rather than causal uplift, guaranteed ROI, or an optimal budget allocation.

- Marketing next step: prioritize measured, randomized TV and Radio experiments, including an appropriate Newspaper comparison, to test the observed hypotheses.
- Collect dated spend, verified units, channel costs, revenue, segment labels, and outcomes. Predefine evaluation metrics and an adequate validation period.
- Use temporal validation once dated records are available. This dataset alone does not justify reallocating a real advertising budget.

5 Limitations and next steps

Only 200 observations and one 40-row holdout are available. Model selection can make the winning CV score optimistic; the test set provides an independent check within this sample. Unmeasured confounding, correlated channels, distribution shift, and absent time information limit marketing interpretation. No forecast for a named future period, causal budget effect, or assured business return is claimed. Verify dataset provenance, redistribution permission, and measurement definitions before production use.

6 Reproducibility

Validation recorded on 19 October 2026: the delivered notebook executed 5 code cells with 0 recorded errors. The analysis pipeline and its built-in data/evaluation checks completed successfully.

Install the pinned requirements and rerun from the repository root. The notebook calls the same analysis pipeline.

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python analysis.py
```

outputs/metrics.json records source SHA-256, model settings, row split IDs, CV results, and held-out errors. Tables retain descriptive statistics, correlations, fold-level CV scores, training-only linear associations, and hypothetical inputs. heldout_predictions.csv records observed and predicted test values. Five PNG charts are generated; optional --data and --output-dir arguments support alternate local paths.